



**Faculty of Software Engineering and Computer Systems**

# **Programming**

Lecture #4.  
SOLID, X-classes

Instructor of faculty  
Pismak Alexey Evgenievich  
Kronverksky Pr. 49, 1331 room  
[pismak@itmo.ru](mailto:pismak@itmo.ru)

Saint-Petersburg

# S.O.L.I.D. принципы

- Принцип единственной обязанности
- Принцип открытости/закрытости
- Принцип подстановки Барбары Лисков
- Принцип разделения интерфейса
- Принцип инверсии зависимостей

# SOLID (Single Responsibility Principle)

```
1.  class Person{
2.      public void eat(){ };
3.      public void walk(){ };
4.      public void run(){ };
5.      public void driveCar(Car car){ };
6.      public void createOcean() {};
7.      public void beSomething() {};
8.  }
```

# SOLID (Open-Closed principle)

```
1.  class Animal {  
2.      private int speed;  
3.  
4.      public int  getSpeed() {  };  
5.      public void setSpeed() {  };  
6.      public void run()  {};  
7.  }  
8.
```

\* в хорошо спроектированных программах новая функциональность вводится путем добавления нового кода, а не изменением старого, уже работающего

# SOLID (Liskov Substitution Principle)

```
1. class Duck {  
2.     public int swim() { }  
3. }
```

```
1. class ToyDuck extends Duck {  
2.     private Battery battery = ...  
3.     public int swim() {  
4.         if (battery.isCharged())  
5.         }  
6. }
```

```
1. Duck[] ducks = // init array different ducks  
2.  
3. for(Duck duck : ducks) {  
4.     duck.swim();  
5. }
```

# **SOLID (Interface Segregation Principle)**

// 404

# SOLID (Dependency Inversion Principle)

```
1.  public class Car {  
2.  
3.      private Wheel wheel = new Wheel();  
4.  
5.      public void go() {  
6.          wheel.rotate();  
7.      }  
8.  
9.  }
```

# SOLID (Dependency Inversion Principle)

```
1.  public class Car {  
2.  
3.      private Wheel wheel;  
4.  
5.      public Car(Wheel wheel) {  
6.          this.wheel = wheel;  
7.      }  
8.  
9.      public void go() {  
10.         wheel.rotate();  
11.     }  
12.  
13. }
```



# SOLID (Dependency Inversion Principle)

```
1.  class SportWheel extends Wheel {
2.      @Override
3.      public void rotate() {
4.          // burn ground
5.      }
6.  }

1.  public static void main(String[] args) {
2.
3.      Car car = new Car(new Wheel());
4.      Car sportcar = new Car(new SportWheel());
5.
6.  }
```

# Вложенные классы

```
public class Car {
```

```
    private Wheel backRightWheel;
```

```
    private Wheel backLeftWheel;
```

```
    private void crash() {}
```

```
    public class Wheel { // доступны все модификаторы доступа
```

```
        public void rotate(float angle) {
```

```
            // ....
```

```
        }
```

```
        public void crash() {
```

```
            // ....
```

```
        }
```

```
    }
```

```
}
```

# Вложенные классы

// создание экземпляра внешнего класса

```
Car car = new Car();
```

// Создание экземпляра внутреннего класса

```
Car.Wheel wheel = car.new Wheel();
```

# Вложенные классы

```
public class Car {
```

```
    private Wheel backRightWheel;  
    private Wheel backLeftWheel;
```

```
    private void crash() {}
```

```
    public class Wheel {
```

```
        public void rotate(float angle) {  
            // ....  
        }
```

```
        public void crash() {  
            // ....  
        }
```

```
    }
```

```
}
```

Пусть поведение машины  
“ломаться” будет закрытым

А колесо можно повредить чем-то  
снаружи, и по этой причине сделаем  
это поведение открытым

Как тогда из метода `Wheel.crash()` вызвать метод `Car.crash()`?

# Вложенные классы

```
public class Car {
```

```
    private Wheel backRightWheel;
```

```
    private Wheel backLeftWheel;
```

```
    private void crash() {}
```

```
    public class Wheel {
```

```
        public void rotate(float angle) {
```

```
            // ....
```

```
        }
```

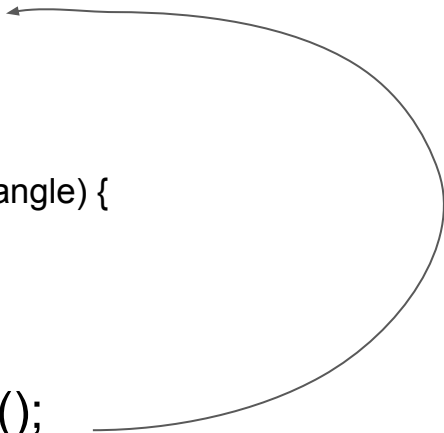
```
        public void crash() {
```

```
            Car.this.crash();
```

```
        }
```

```
    }
```

```
}
```



# Внутренние классы

```
public class Car {
```

```
    // ....
```

```
    public static class BadAir {
```

```
        public void generate() {
```

```
            // ....
```

```
        }
```

```
    }
```

```
}
```

# Внутренние классы (static)

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Car.Wheel wheel = new Car.Wheel();  
  
    }  
  
}
```

# Локальные классы

```
public class Main {  
  
    public static void main(String[] args) {  
  
        class TheBestPlaceForUselessClassDeclaration {  
            // ....  
        }  
        TheBestPlaceForUselessClassDeclaration tbpfucd =  
            new TheBestPlaceForUselessClassDeclaration();  
    }  
}
```



# Локальные классы 2

```
public class Main {  
  
    public static void main(String[] args) {  
  
        if (args.length == 0) {  
            class TheBestPlaceForUselessClassDeclaration {  
                // ....  
            }  
            TheBestPlaceForUselessClassDeclaration tbpfucd =  
                new TheBestPlaceForUselessClassDeclaration();  
        }  
  
    }  
  
}
```

# Локальные классы 3

```
public class Main {  
  
    public static void main(String[] args) {  
  
        while (true) {  
            class TheBestPlaceForUselessClassDeclaration {  
                // ....  
            }  
            TheBestPlaceForUselessClassDeclaration tbpfucd =  
                new TheBestPlaceForUselessClassDeclaration();  
        }  
    }  
}
```

# Локальные классы 4

```
public class Main {  
  
    public static void main(String[] args) {  
  
    }  
  
    // блок инициализации  
    {  
        class TheBestPlaceForUselessClassDeclaration {  
            // ....  
        }  
        TheBestPlaceForUselessClassDeclaration tbpfucd =  
            new TheBestPlaceForUselessClassDeclaration();  
    }  
}
```

}

# Локальные классы

- Модификатор доступа не указывается
- Невозможно объявление статических методов (и любых иных статических членов), но
- Возможно использование статических констант
- Захват внешних локальных переменных возможен, если они определены, как **effectively final**
- Не могут быть статическими

# Анонимные классы

```
public interface Runnable {  
    void run();  
}
```

```
public class Runner {  
  
    public void start (Runnable instance) {  
        instance.run();  
    }  
  
    public void test() {  
  
        // тут хочется вызвать метод start, но реализацию  
        // Runnable писать лениво и нет необходимости  
  
    }  
  
}
```

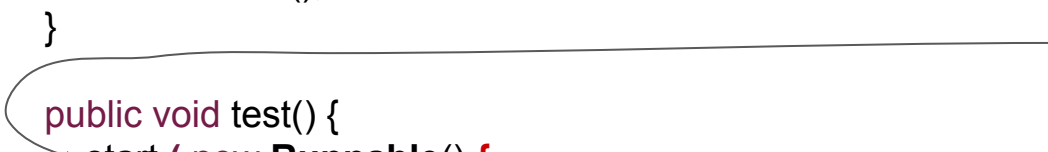
# Анонимные классы

```
public interface Runnable {  
    void run();  
}
```

```
public class Runner {
```

```
    public void start (Runnable instance) {  
        instance.run();  
    }
```

Вызов метода start



```
    public void test() {  
        start ( new Runnable() {
```

```
            public void run() { }
```

```
        } );
```

```
    }
```

```
}
```

# Анонимные классы

```
public interface Runnable {  
    void run();  
}
```

```
public class Runner {
```

```
    public void start (Runnable instance) {  
        instance.run();  
    }
```

```
    public void test() {  
        start ( new Runnable() {  
            public void run() { }  
        } );  
    }
```

```
}
```

Вызов метода start

Создание объекта Runnable на лету и передача его в качестве параметра методу start

# Анонимные классы

```
public interface Runnable {  
    void run();  
}
```

```
public class Runner {
```

```
    public void start (Runnable instance) {  
        instance.run();  
    }
```

```
    public void test() {  
        start ( new Runnable() {
```

```
            public void run() { }
```

```
        } );
```

```
    }
```

```
}
```

Переопределение run,  
объявленного в  
интерфейсе Runnable





# Почему анонимные?

```
public interface Runnable {  
    void run();  
}
```

```
public class Runner {  
  
    public void start (Runnable instance) {  
        instance.run();  
    }  
  
    public void test() {  
        start ( new Runnable() {  
  
            public void run() { }  
  
        } );  
    }  
}
```

Анонимный класс - это аналог локального класса. Разница только в наличии у класса имени для повторного использования

```
public void test() {
```

```
    class X implements Runnable {  
        public void run() { }  
    };  
    start ( new X() );  
}
```



# Анонимные классы

```
public class Car {  
    void go() { }  
}
```

```
public class Main {
```

```
    public static void main(String[] s) {  
        start ( new Car() {
```

```
        {  
            // инициализация нового объекта  
            // внедряя код в блок инициализации  
        }  
    }  
});
```

```
}
```

```
}
```

Это не обязательно  
интерфейсы

возможно не только переопределение  
методов, но и использование блоков  
инициализации, “**добавление**” полей и  
методов

# Анонимные классы

```
public class Car {  
    void go() { }  
}
```

```
public class Main {
```

```
    public static void main(String[] s) {  
        start ( new Car() {
```

```
            {  
                // инициализация  
                // нового объекта  
                // внедряя код в блок  
                // инициализации  
            }  
        }  
    }  
};
```

```
    }  
}
```

```
}
```

```
public static void main(String[] s) {
```

```
    class X extends Car {
```

```
        {  
            // инициализация  
            // нового объекта  
            // внедряя код в блок  
            // инициализации  
        }  
    }  
};
```

```
    start ( new X() );  
}
```

```
}
```



# Record'ы

```
public class Animal {

    private String name;

    public Animal(String name) {
        this.name = name;
    }

    public String getName() {
        return this.name;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;

        Animal animal = (Animal) o;

        return Objects.equals(name, animal.name);
    }
}
```

```
@Override
public int hashCode() {
    return name != null ? name.hashCode() : 0;
}

@Override
public String toString() {
    return "Animal{" +
        "name='" + name + '\\'' +
        '}' ;
}
```

# Record'ы

```
public record Animal(String name){}
```

- Record не может наследоваться от какого-нибудь класса, но может реализовывать интерфейсы
- У Record не может быть других полей объекта, кроме тех, которые объявлены в конструкторе при описании класса (да, это конструктор по умолчанию, кстати). Статические — можно.
- Поля неявно являются финальными. Объекты неявно являются финальными. Со всеми вытекающими, вроде невозможности быть абстрактными.
- От Record нельзя наследоваться, т.к. он “final class”